

Re-implementation of Scikit-Learn DBSCAN in R

Brian Ge, Fan Zhang and Grace Richards

December 15, 2024

Contents

1	Introduction	2
2	Description of Problem	2
3	Explanation of Algorithm	3
3.1	DBSCAN Algorithm	3
3.1.1	Key Parameters	3
3.1.2	Steps	3
3.2	Implementation	4
3.2.1	Key Features	4
3.2.2	Naive Implementation	4
3.2.3	Sparse Matrix Implementation	4
4	Evaluation of Results	5
4.1	Choice of ε	5
4.2	Different Shapes of Data	5
4.3	Test with Metrics	6
4.3.1	Small Dataset	6
5	Contributions	6
A	Use of GenAI	7
B	GitHub Branches	7

The GitHub Repo is available here. If it does not work, turn to footnote. ¹

¹<https://github.com/cajole-fff/UMich-BIOSTAT615-2024Fall-Project-Group11.git>

1 Introduction

Clustering is a **unsupervised** machine learning technique used to group similar data points based on their characteristics. It is widely used in areas like market segmentation, image segmentation, anomaly detection and document organization. Popular cluster algorithms include K-means, hierarchical clustering, DBSCAN, Gaussian mixture model and etc. Figure 1 is a picture showing different clusters of data.

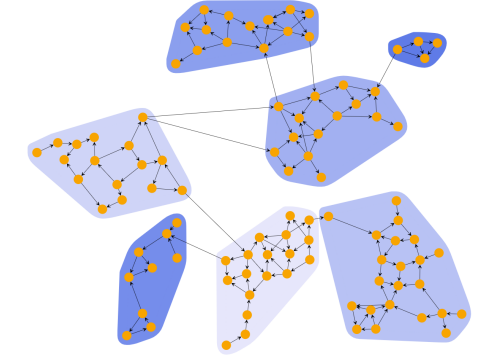


Figure 1: Clustering. [1]

Table 1 shows a comparison of different clustering algorithms. We can see that DBSCAN is robust to outliers and does not assume spherical clusters. It also does not require specifying the number of clusters.

Table 1: Comparison of Comment Clustering Algorithms.

Algorithm	Handles Noise	Cluster Shape
DBSCAN	Yes	Arbitrary
K-Means	No	Spherical
Hierarchical	No	Various
Gaussian Mixture	No	Elliptical
Bernoulli Mixture	No	Binary Data Clusters

In this project, we focused on reimplementing Scikit-Learn’s DBSCAN (Density-Based Spatial Clustering of Applications with Noise) algorithm in R. Our primary goal was to achieve functional parity with Scikit-Learn’s implementation while leveraging the performance advantages of Rcpp.

By supporting both Euclidean and Minkowski distance metrics, the implementation accommodates datasets with varying distributions, making the algorithm adaptable to diverse clustering tasks. Users can further tailor the clustering process using parameters such as the Minkowski parameter (p), improving the accuracy and applicability of the results across domains in high-dimensional feature clustering.

2 Description of Problem

Currently, the `dbscan` package in R can help to fit DBSCAN models. However, it is somewhat different from the `dbscan` class in Python `sklearn` package. For example, it does not have build-in Manhattan distance and the user should self-define it if needed. Moreover, it does not support sparse matrix, which contributes to the inefficiency in many circumstances. There are

many real-life sparse matrices. For example, we may obtain data from a user rating system and use it to find the similarities between different users. Since many users only rate a few products, the dataset will be a sparse matrix. Utilizing the features of sparse matrix in this case can improve the efficiency.

Moreover, if people need to visualize the DBSCAN fitting result, they need to call other plotting packages like `ggplot2`. It is not so convenient and we plan to integrate the plotting functions into the package.

3 Explanation of Algorithm

3.1 DBSCAN Algorithm

3.1.1 Key Parameters

There are two key parameters in DBSCAN algorithm: [2]

eps (ε) A distance measure that will be used to locate the points in the neighborhood of any point.

minPts The minimum number of points (a threshold) clustered together for a region to be considered dense.

It is a *density-based* clustering algorithm which groups points that are closely packed together and marks points in low-density regions as outliers. It mainly considers two important concepts:

Reachability in terms of density establishes a point to be reachable from another if it lies within a particular distance (eps) from it. [2]

Connectivity involves a transitivity based chaining-approach to determine whether points are located in a particular cluster. [2]

3.1.2 Steps

Below is the steps of DBSCAN algorithm.[2]

1. Mark all points as unvisited.
2. Start with an unvisited point P .
 - (a) Mark P as visited.
 - (b) Find all points within ε -neighborhood of P :
 - total number of points $> \text{minPts} \Rightarrow$ mark P as *core point* and expand the cluster.
 - total number of points $< \text{minPts} \Rightarrow$ mark P as *noise point*.
3. Repeat expanding cluster for each point in it until the cluster cannot be expanded.
4. Return to step 2. and proceed to the next unvisited point until all points are visited.

Points are categorized as:

- Core points: Sufficient neighbors within ε
- Border points: Insufficient neighbors but within ε of a core point
- Noise points: Neither core nor border points

3.2 Implementation

3.2.1 Key Features

In this project, we implement an *object-oriented* class DBSCAN which is a `RR6::R6Class`, and every function keeps consistency with `sklearn.cluster.DBSCAN` in Python. When the initialization function is called, it will return a `R6Class` object with parameter as set. Parameters include `eps`, `min_samples`, `metric`, `metric_samples`, `algorithm`, `leaf_size`, `p`, `n_jobs` and `max_memory`. The parameter `max_memory` is added as a critical value. If `object.size(X) < max_memory`, it will store the matrix for convinence. This parameter prevents high memory usage.

As mentioned before, we also added metric evaluation functions Silhouette score and adjusted rand index, and also visulization functions like plot clusters and plot core samples. The user doesn't need to input the matrix again when they call these functions. Moreover, if they need to calculate the metrics or generate plots for other datasets, they can simply use those functions as tool functions by initializing the `R6Class` as a singleton.

After initializing, there is no setter functions for parameters to prevent changes.

3.2.2 Naive Implementation

A key focus of our reimplementaion was to optimize computationally intensive operations such as neighborhood searches. The following features were introduced to enhance performance:

- Multithreading Support: The `n_jobs` parameter allows users to specify the number of threads, significantly reducing computation time on large datasets when using multiple CPU cores.
- Evaluation: To ensure a comprehensive assessment of clustering results, we included two evaluation metrics:
 - Silhouette Score measures the compactness and separation of a cluster, providing valuable insights into unsupervised cluster tasks.
 - Adjusted Rand Index (ARI): Evaluates the alignment between predicted clusters and ground-truth labels, offering robustness in supervised scenarios. These metrics complement the core algorithm, enabling users to assess and refine clustering quality effectively.

3.2.3 Sparse Matrix Implementation

The current implementation of DBSCAN in Python's Scikit-Learn package does not support sparse matrix formatting. There are many real-world applications in which we use sparse data matrices, such as text mining and natural language processing, spatial epidemiology, genetics, genomics, and imaging" (Topic 3 Slides). In such cases, the usage of sparse matrix data structures can greatly streamline the analysis.

One advantage of this reimplementaion of DBSCAN in R is its compatibility with sparse matrices using R's 'Matrix' package. This package's sparse matrix data types ignore the zero-entries of these matrices in order to store data much more compactly than a standard R matrix with all zero and non-zero entries included. There are two main efficiency gains that come from the use of sparse matrix data types: one, that computations using sparse matrix representations use much less storage space, and two, that these computations can be much faster than that of base-R matrices. Both of these efficiency gains have the potential to be very valuable when working with large datasets.

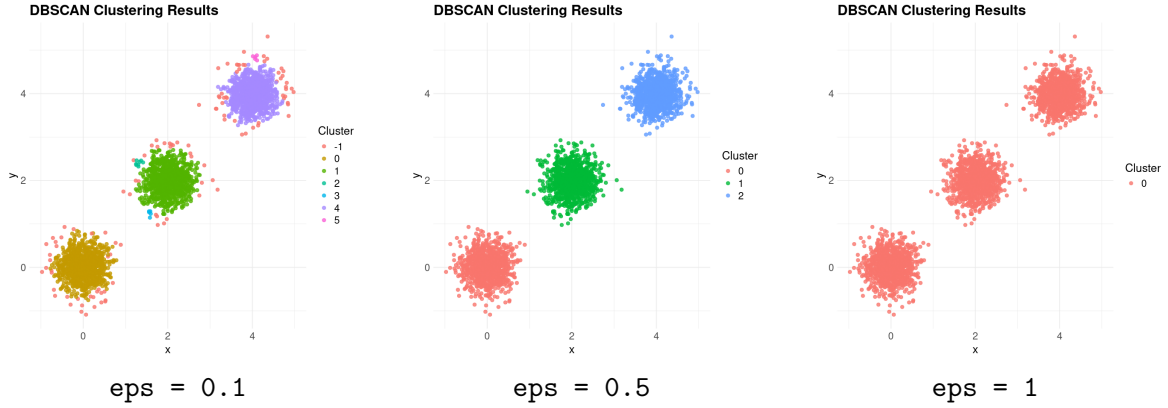
The sparse matrix implementation is mobilized in the calculation of the Minkowski distance matrix within the central clustering algorithm of this R package. Using Compressed Sparse Row representation of the training data matrix, the function bypasses storage of and calculations involving lots of zero values, increasing the efficiency of DBSCAN.

Suppose we have a matrix $A_{n \times p}$ with sparsity S , that is the probability that any given value is nonzero is $(1 - S) \ll 1$. Thus, assuming that the entries of different rows are independent of one another, the probability that the k -th entries of two different rows of A are nonzero is $(1 - S)^2$, a very small value. Thus, sparse matrix implementation of the Minkowski distance is made faster not only by the small number of values provided, but by the fact that the number of points where we calculate a difference is *very* small. In fact, the overwhelming majority of the distance calculation is simply summing the absolute values.

4 Evaluation of Results

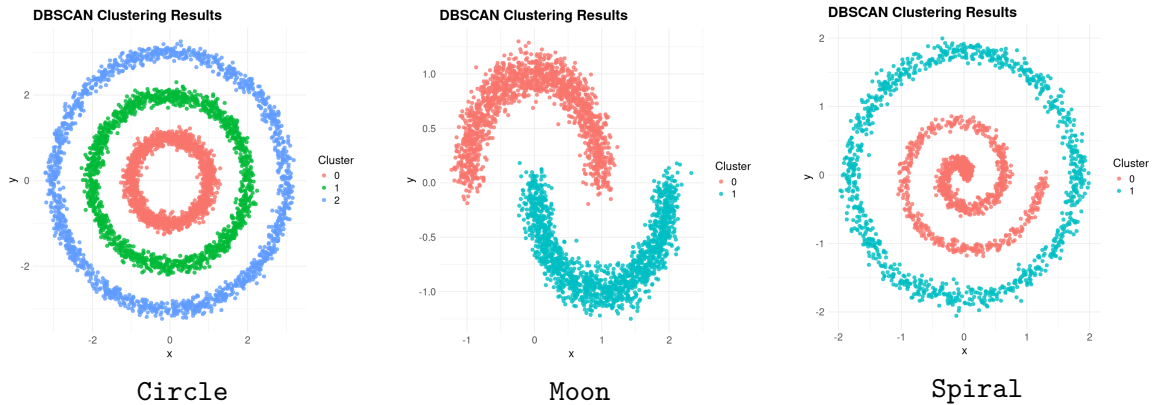
4.1 Choice of ε

In this part, we generate three clusters with mean 0, 2, 4 respectively and standard error 0.3. We fit the data using our DBSCAN model with different choice of **eps**. Below is the result:



We can easily find that the choice of **eps** matters. Only when we choose **eps** = 0.5 it will generate correct result. If the **eps** is too small, points on the edge of a cluster will be classified into a different cluster. If the **eps** is too large, points from different clusters will be classified into a single cluster.

4.2 Different Shapes of Data



4.3 Test with Metrics

4.3.1 Small Dataset

The dataset used in this evaluation consisting of three well-defined clusters, each generated from a multivariate normal distribution. Each cluster contains 100 data points, generated with a specified mean and a diagonal covariance matrix. Cluster 1 centers at (0, 0), Cluster 2 centers at (10, 10), and Cluster 3 centers at (0, 10), ensuring clear separation between clusters. The true cluster labels are predefined, allowing for supervised evaluation metrics like Adjusted Rand Index (ARI). This dataset is ideal for testing DBSCAN because of its clear cluster boundaries and the absence of noise.

The computed ARI is 0.5698, indicating alignment between the DBSCAN clustering results and the truth labels. This suggests that the algorithm successfully identified the general structure of the clusters but with some misclassifications.

The silhouette score is 0.6172, which falls in the range of good clustering quality. A score closer to 1 would represent well-separated clusters, while scores closer to 0 indicate overlapping clusters. This result shows that most data points are appropriately assigned to their clusters.

The noise ratio is 0, which means no points were classified as noise by DBSCAN with the chosen parameters ($\text{eps} = 5$, $\text{samples} = 5$).

The clustering process was executed efficiently, with an average execution time of 5.187 milliseconds per run (across 10 evaluations). The speed demonstrates that implementation of DBSCAN is computationally efficient.

5 Contributions

Table 2: Contribution of Group Members.

Item		Grace	Fan	Brian
Project Proposal			High	
Progress Report			High	
Coding	Structure	Low	High	
	Build & Install		High	
	Fit		High	
	Metrics		High	
	Visualization		High	
	Test		High	Medium
Presentation			High	
Report	Introduction	Medium	High	High
	Description		High	
	Algorithm & Implementation	Medium	Medium	Medium
	Test & Result		High	Medium

References

- [1] yWorks. “Clustering Graphs and Networks.” *yWorks*. Available: <https://www.yworks.com/pages/clustering-graphs-and-networks>. Accessed 03 Dec. 2024.
- [2] Nagesh Singh Chauhan, KDnuggets. “DBSCAN Clustering Algorithm in Machine Learning”. *KDnuggets*, 04 Apr 2022. Available: <https://www.kdnuggets.com/2020/04/dbscan-clustering-algorithm-machine-learning.html>. Accessed 03 Dec. 2024.
- [3] Scikit-Learn. “DBSCAN Documentation”. *Scikit-Learn*. Available: <https://scikit-learn.org/1.5/modules/generated/sklearn.cluster.DBSCAN.html>. Accessed 03 Dec. 2024.

A Use of GenAI

To help us work more efficiently, we use GenAI (mainly ChatGPT by OpenAI) to get some ideas of the project. Below are the chats with ChatGPT:

- 1. <https://chatgpt.com/share/675cab6c6-75d8-8005-99c3-c0151c4076a5>
- 2. <https://chatgpt.com/share/675cf002-f984-8005-8185-9450c34fad70>
- 3. <https://chatgpt.com/share/675cf010-b7f0-8005-b1b0-e057de7eae1f>
- 4. <https://chatgpt.com/share/675cf020-d21c-8005-9aea-eae1c5eb51c0>
- 5. <https://chatgpt.com/share/675cf037-d988-8005-8312-ee7154c94d09>
- 6. <https://chatgpt.com/share/675cf053-b370-8005-ab4a-7181a54eebb7>
- 7. <https://chatgpt.com/share/675d0482-4884-8005-b32d-572c705dcfeb>
- 8. <https://chatgpt.com/share/675d04b7-9974-8005-9bd8-2a3403fdc0d4>

B GitHub Branches

Below are the branches and commit history of our project:

Fan (GitHub `cajole-fff`) master, fzhang

Grace (GitHub `grichards-beepboop`) grich

Brian (GitHub `briange90`) test